

Proyecto gimnasio

Ingenieria de Software II

Facundo Lella
Renzo Dávila
Ramiro Martinez
Alejandro Moreno

[Link al repositorio](#)



Tabla de contenidos

01

**Inicialización:
Planteo del problema**

02

**Elaboracion:
Diagramas de diseño**

03

**Elaboracion:
Caso de uso critico**

04

**Elaboracion:
Requisitos no funcionales**

05

**Implementacion:
Patrones utilizados**

06

**Mercado y posible
nueva implementación**



01

Planteo del problema

Introducción



Situacion planteada

El Gimnasio Sport contrata nuestros servicios con el fin de diseñar un sistema administrativo que los ayude en las tareas del día a día.



RUP

El Proceso Unificado Racional es una metodología de desarrollo iterativa que organiza el trabajo en cuatro fases: Iniciación, Elaboración, Construcción y Transición.

Iniciación

Se define el alcance y objetivos del proyecto a desarrollar. En esta etapa se captan los requisitos principales del sistema y se comprende el negocio. Además se pueden realizar algunas estimaciones preliminares de tiempo y costo de desarrollo

Administración de usuarios

Empleados y socios

Envío de mensajes

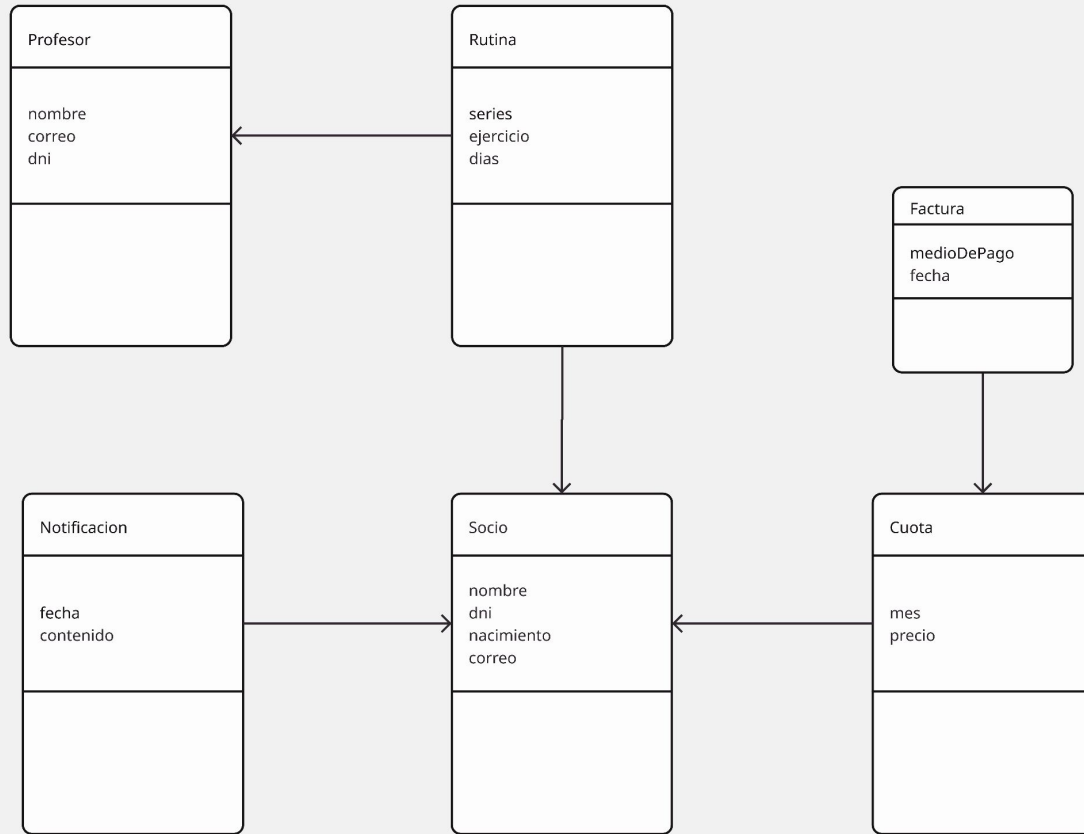
Saludos y avisos

Pagos incorporados

Uso de mercado pago

Seguimiento de rutinas

Ejercicios a realizar por los socios



02

Diagramas de diseño



Elaboracion

En esta fase se definen los requisitos clave y se diseña la arquitectura base del sistema, reduciendo riesgos y estableciendo una estructura sólida para la construcción.

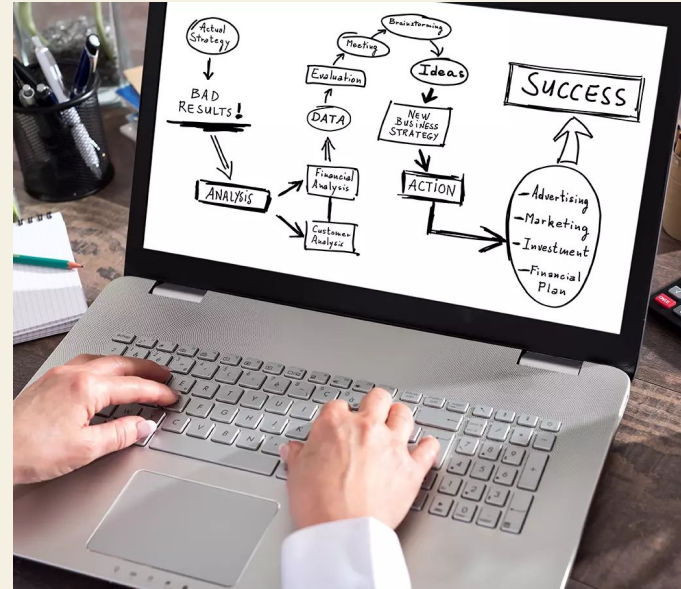


Diagrama de casos de uso

Diagrama entidad relación

[Link](#)

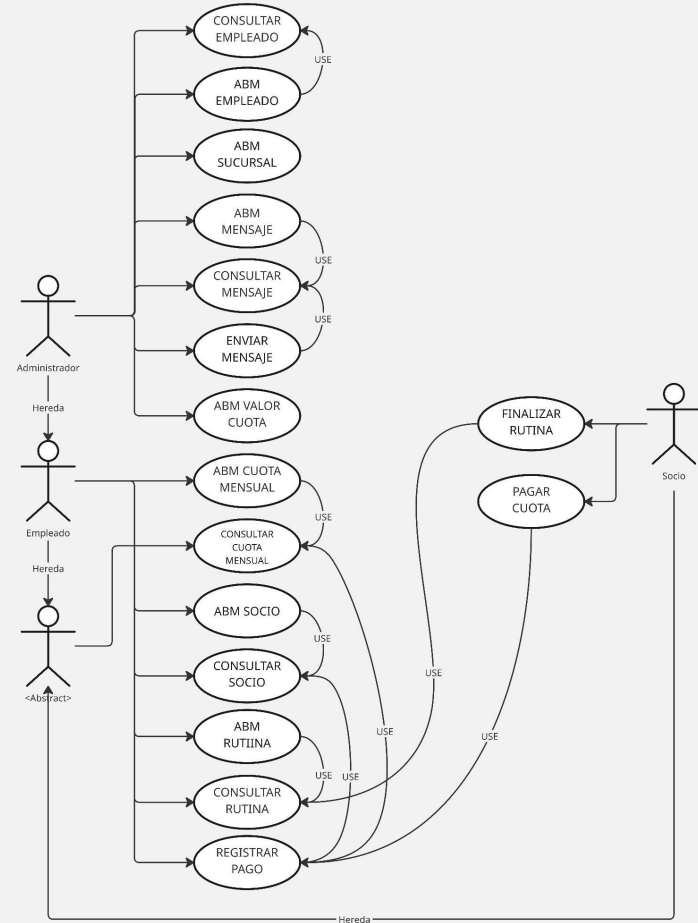
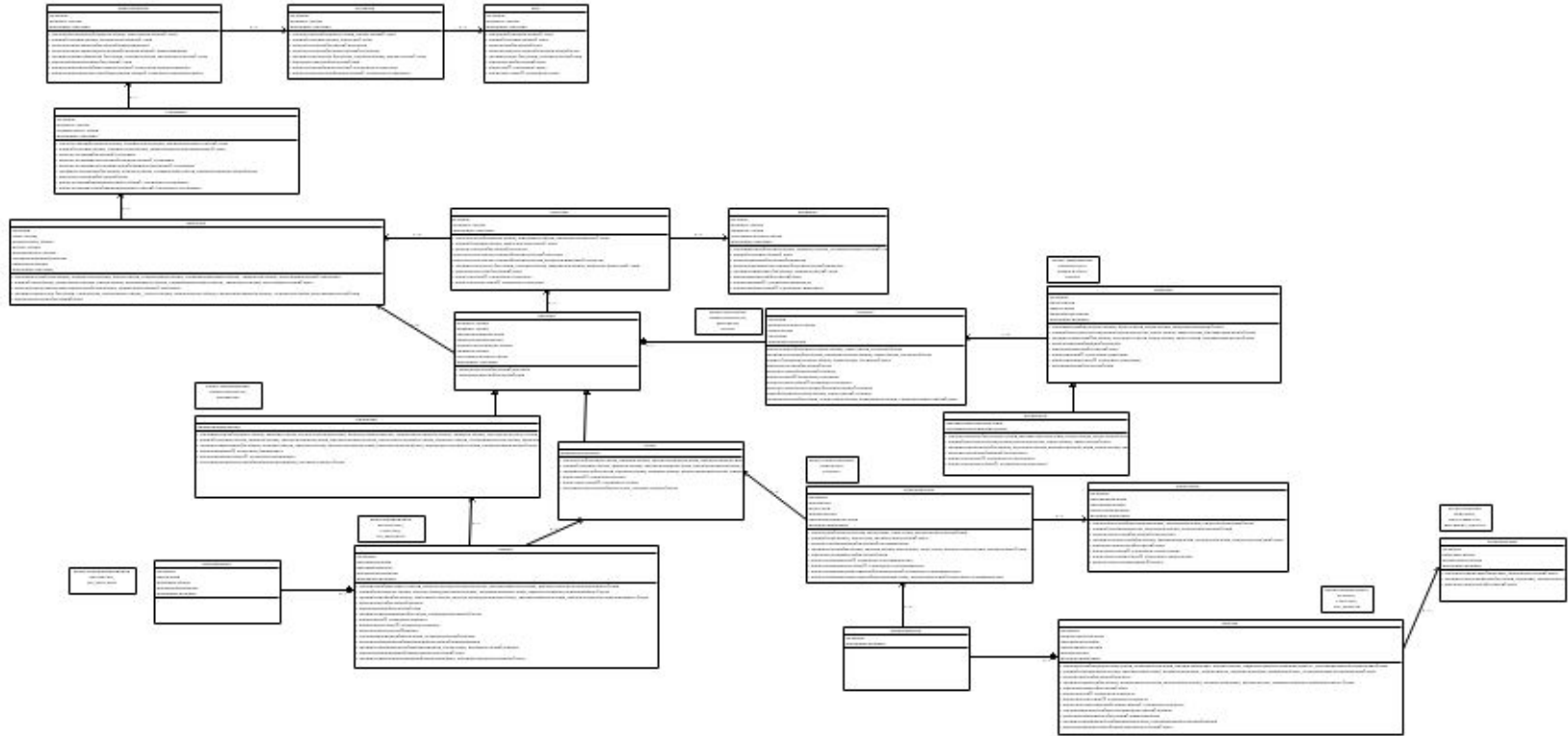


Diagrama de clases de diseño





03

Caso de uso critico



Envío de mensajes:

Saludo de cumpleaños

El sistema deberá poder enviar un mensaje a todos aquellos socios que cumplan años el día de hoy

Aviso de pago de cuota

El sistema deberá poder enviar un mensaje a todos aquellos clientes que su cuota mensual ya está disponible para su pago

Aviso de deuda

El sistema deberá poder enviar un mensaje a todos aquellos socios que tengan cuotas vencidas que no hayan sido pagadas

Escenario de caso de uso

Link

Diagrama de secuencia

Link





Lista de mensajes



Acciones	Titulo	Tipo	Fecha envio	Usuarios enviados	Accion
	¡Promocion de enero!	PROMOCION	27/04/2025	437	<button>Enviar</button>
	¡Muy feliz cumpleaños!	CUMPLEAÑOS	31/02/2025	3	<button>Enviar</button>



Editar mensaje

Título

¡Feliz cumpleaños para ti!

Texto del mensaje

De parte del equipo de GimnasioSport, ¡te deseamos un excelente día y te estaremos esperando para festejar con disciplina y voluntad!

Tipo del mensaje

CUMPLEAÑOS



Seleccione una fecha



Septiembre 2025



S	M	T	W	T	F	S
1	2	3	4	5	6	7
8	9	10	11	12	13	14
15	16	17	18	19	20	21
22	23	24	25	26	27	28
29	30	31	1	2	3	4

[Seleccionar el día de hoy](#)

Aceptar

Cancelar

04

Requisitos no funcionales



Rendimiento

El sistema deberá permitir que los socios puedan acceder a su rutina y estado de pagos en menos de 3 segundos desde que se ingresa la solicitud.

Usabilidad

La interfaz deberá ser sencilla e intuitiva, permitiendo que un nuevo usuario pueda comprender su uso básico en menos de 15 minutos.

Disponibilidad

La aplicación deberá estar disponible todos los días a todas horas para permitir a los usuarios visualizar sus responsabilidades.

Seguridad

El sistema deberá autenticar a todos los usuarios mediante credenciales seguras, y los datos de pagos deberán transmitirse en forma cifrada.

Escalabilidad

El sistema deberá soportar un incremento en la cantidad de socios de hasta un 50 % sin necesidad de rediseñar la arquitectura base.

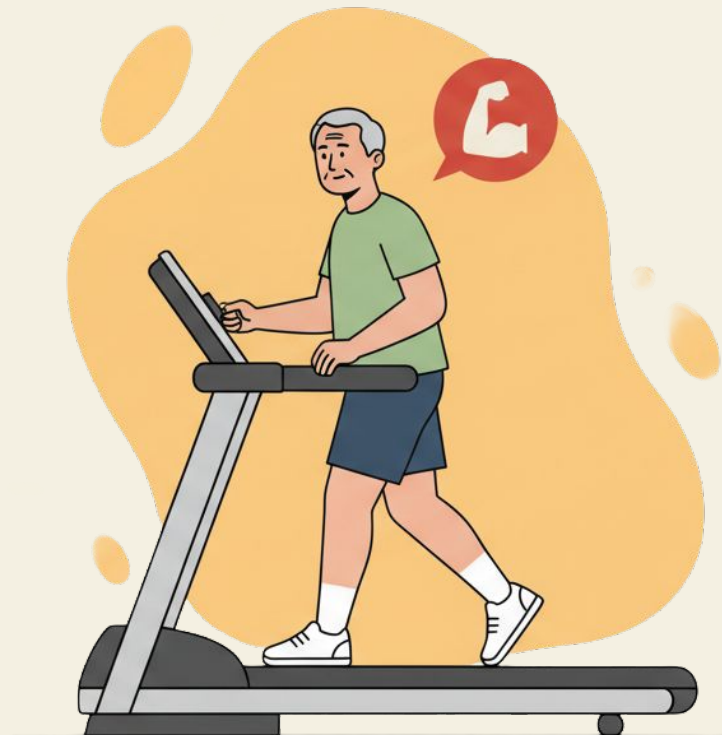
Mantenibilidad

El sistema deberá estar documentado y estructurado para facilitar la corrección de errores y la incorporación de nuevas funcionalidades.



05

Patrones utilizados



1. Capas

El patrón en capas organiza una aplicación dividiéndola en niveles con responsabilidades bien definidas.

Cada capa interactúa únicamente con la adyacente, lo que facilita el mantenimiento, la reutilización y el bajo acoplamiento.

Cliente		Servidor		
Capa de Presentación		Capa de Negocio		
Capa Cliente	Capa Controlador	Capa Logica de Negocio	Capa Dominio	Capa Persistencia
Front End		Back End		

En nuestro caso:

- Controladores: gestionan las peticiones del usuario.
- Servicios: contienen la lógica de negocio (por ejemplo, el cálculo de cuotas o manejo de facturas).
- Repositorios (DAO): se encargan del acceso a la base de datos.
- Entidades (Modelo): representan los objetos del dominio, como Socio, Empleado o Factura.

2. DAO

El patrón DAO (Data Access Object) separa la lógica de negocio del acceso a datos, creando una capa específica para interactuar con la base de datos.

De esta forma, el resto de la aplicación no necesita conocer detalles de SQL ni de la persistencia.

En nuestro caso:

- Las interfaces en repository/ (ejemplo: SocioRepository, FacturaRepository) actúan como DAOs, extendiendo JpaRepository.
- Esto permite realizar operaciones CRUD sobre las entidades sin acoplar la lógica de negocio a la base de datos.

```
1
2 @Repository
3 public interface SocioRepository extends JpaRepository<Socio, String> {
4
5     @Query("SELECT COUNT(s.id) FROM Socio s")
6     public Long obtenerProximoNumeroSocio();
7
8     @Query("SELECT s FROM Socio s WHERE FUNCTION('MONTH', s.fechaNacimiento) = :mes AND FUNCTION('DAY', s.fechaNacimiento) = :dia")
9     public Collection<Socio> listarCumpleaneros(@Param("dia") int dia, @Param("mes") int mes);
10
11     @Query("SELECT s FROM Socio s WHERE s.eliminado = false")
12     public Collection<Socio> listarSociosActivos();
13
14     @Query("SELECT DISTINCT s FROM Socio s JOIN s.cuotasMensuales c WHERE c.estado = :estado")
15     public Collection<Socio> SocioConDeudas(@Param("estado") EstadoCuotaMensual estado);
16 }
```

3. Experto en información

El patrón Experto en información asigna la responsabilidad de manejar datos y operaciones al objeto que posee la información necesaria.

De esta manera, cada clase del modelo conoce y gestiona sus propios atributos, evitando depender de otras para tareas básicas.

En nuestro caso:

- Factura almacena sus detalles y estados, siendo responsable de su propia información.
- Rutina almacena sus ejercicios y los consulta.

```
1 public DetalleRutina crearDetalleRutina(String idRutina, LocalDate fecha, String actividad) {
2     if (fecha == null || actividad == null || actividad.isBlank()) {
3         throw new BusinessException("Fecha y actividad son obligatorias");
4     }
5     Rutina rutina = buscarRutina(idRutina);
6     DetalleRutina detalle = new DetalleRutina();
7     detalle.setFecha(fecha);
8     detalle.setActividad(actividad.trim());
9     detalle.setEstadoDetalle(EstadoDetalleRutina.SIN_REALIZAR);
10    detalle.setRutina(rutina);
11    detalle = detalleRutinaRepository.save(detalle);
12    rutina.getDetalles().add(detalle);
13    return detalle;
14 }
```

4. Inyección de dependencia

La Inyección de Dependencias (DI) es un principio que permite que los objetos reciban sus dependencias desde el exterior, en lugar de crearlas directamente.

Esto logra bajo acoplamiento, mayor flexibilidad y facilidad para realizar pruebas.

En nuestro caso:

Spring inyecta automáticamente servicios y repositorios mediante `@Autowired`.

Ejemplo: un Controller recibe su Service sin necesidad de usar `new`.

```
1 public class MensajeController {  
2  
3     @Autowired  
4     private MensajeService service;  
5  
6     @Autowired  
7     private PromocionService promocionService;  
8  
9     @Autowired  
10    private UsuarioService usuarioService;
```

5. DTO

Un DTO (Data transfer object) es un objeto usado para transportar datos entre capas o servicios, evitando exponer directamente las entidades del dominio.

Ayuda a mantener bajo acoplamiento y mayor seguridad en la comunicación.

En nuestro caso:

Usamos MercadoPagoPreferenceRequest para intercambiar información con la API de MercadoPago.

```
1 public record MercadoPagoPreferenceRequest(  
2     @NotBlank String title,  
3     String description,  
4     @NotNull @Positive Integer quantity,  
5     @NotNull @DecimalMin(value = "0.01") BigDecimal unitPrice,  
6     String currencyId,  
7     String externalReference,  
8     String successUrl,  
9     String failureUrl,  
10    String pendingUrl,  
11    String autoReturn,  
12    String notificationUrl) {  
13 }
```

06

Mercado y posible implementación



Algunos competidores en el mercado

Glofox

Software de gestión para gimnasios y franquicias, con funciones de administración de miembros, pagos recurrentes, clases, entre otros.

GymMaster

Permite llevar registros detallados de los socios, membresías, comunicarse en masa (por ejemplo, envío de mensajes), reservar clases y gestionar ventas dentro del gimnasio.

Perfect Gym

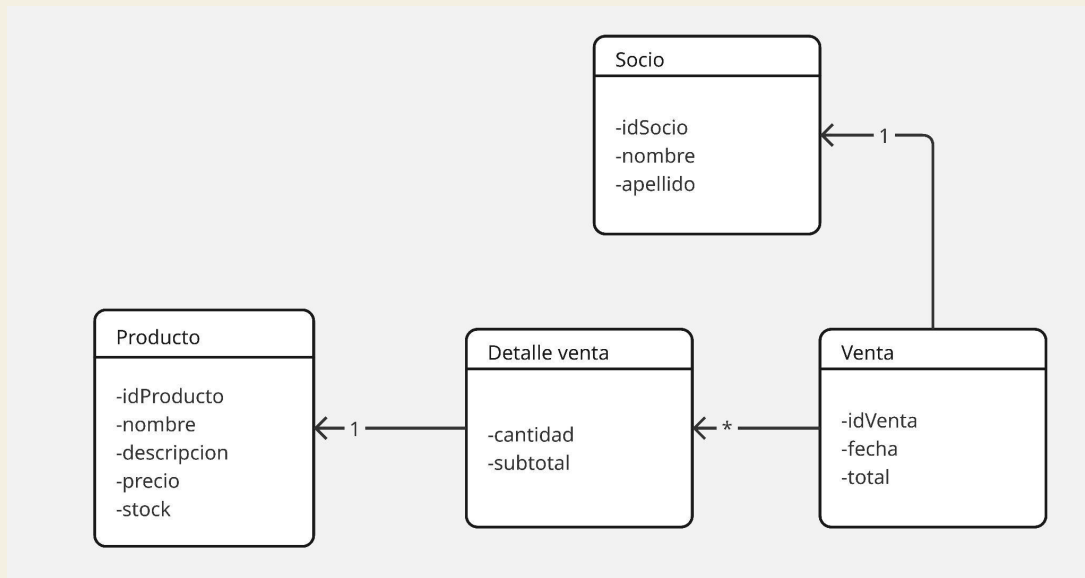
Plataforma para cadenas y gimnasios que ofrece gestión de membresías, cobros automatizados, comunicación personalizada con los miembros, y otras funcionalidades financieras.

ClubOS

Ofrece software de gestión para gimnasios, incluyendo módulos de membresía, ingresos recurrentes, marketing, comunicación, etc.

Implementacion: Servicio de ventas

Se propuso un módulo de punto de venta que permite registrar compras de socios, controlar stock de productos y guardar las ventas realizadas en la base de datos.



¡Gracias!



CREDITS: This presentation template was created by [Slidesgo](#), and includes icons by [Flaticon](#) and infographics & images by [Freepik](#)